

COOCHBEHAR GOVERNMENT ENGINEERING COLLEGE

NAME: MURSHID ALAM

UNIVERSITY ROLL:34900324033

SEMESTER:5TH YEAR: 3RD

DEPARTMENT: ELECTRONICS AND COMMUNICATION ENGINEERING

DIGITAL COMMUNICATION LABORATORY

Experiment 6 — Digital Line Coding and Power Spectral Density

•**AIM:** Generate common line codes from the same bit sequence, estimate normalized PSD using Welch's method, compare DC content, bandwidth and self-clocking capability, measure average level and running digital sum, and test a long run of identical bits.

Bit sequence for mandatory 8-bit validation: 10110010

•Theory:

Digital line coding converts binary data into a baseband waveform suitable for transmission.

The important properties are **DC component**, **required bandwidth**, **transition density**,

and **self-clocking capability**. A good code should provide reliable timing recovery while

using bandwidth efficiently and avoiding a large DC component. **Unipolar NRZ:** binary 1 is represented by +A and 0 by 0. It is simple but has a strong DC component for data with more ones and poor clock recovery during long identical runs. **Polar NRZ-L:** 1 = +A and 0 = -A. It has zero average for balanced data, but long identical bits can produce no transitions. **Polar RZ:** each symbol returns to zero during the second half of the bit. This improves timing information but occupies more bandwidth than NRZ. **Manchester:** every bit contains a mid-bit transition. In this experiment, 1 is low-to-high and 0 is high-to-low. It is self-clocking and has no DC component for ideal balanced operation, but needs comparatively high bandwidth. **Differential Manchester:** there is always a mid-bit transition; the information is represented by the presence/absence of a transition at the beginning of the bit. It is self-clocking and polarity-insensitive. **AMI:** binary 0 is represented by zero voltage and successive binary 1s

alternate between +A and -A. This strongly reduces DC content and provides useful transitions when 1s occur, but a long run of zeros has no transitions.

MATLAB CODE:

%% Experiment 6 – Digital Line Coding and Power Spectral Density

% Digital Communication Laboratory

% Sequence used for mandatory validation: 10110010

Clear; close all; clc;

Bits = [1 0 1 1 0 0 1 0];

A = 1;

Tb = 1;

Spb = 100; % samples per bit

Fs = spb/Tb;

T = (0:length(bits)*spb-1)/fs;

%% Generate six line codes

unipolarNRZ = zeros(1,length(bits)*spb);

polarNRZ = zeros(1,length(bits)*spb);

polarRZ = zeros(1,length(bits)*spb);

manchester = zeros(1,length(bits)*spb);

diffMan = zeros(1,length(bits)*spb);

AMI = zeros(1,length(bits)*spb);

Idx = 1;

amiPol = A;

dmLevel = A;

for k = 1:length(bits)

b = bits(k);

r = idx:idx+spb-1;

% Unipolar NRZ

unipolarNRZ® = b*A;

% Polar NRZ-L

polarNRZ® = (2*b-1)*A;

% Polar RZ: first half pulse, second half zero

polarRZ(idx:idx+spb/2-1) = (2*b-1)*A;

polarRZ(idx+spb/2:idx+spb-1) = 0;

% Manchester: 1 = low-to-high, 0 = high-to-low

If b == 1

Manchester(idx:idx+spb/2-1) = -A;

Manchester(idx+spb/2:idx+spb-1) = A;

Else

Manchester(idx:idx+spb/2-1) = A;

Manchester(idx+spb/2:idx+spb-1) = -A;

End

% Differential Manchester:

% 0 = transition at beginning; 1 = no beginning transition

If b == 0

dmLevel = -dmLevel;

end

diffMan(idx:idx+spb/2-1) = dmLevel;

dmLevel = -dmLevel; % mandatory mid-bit transition

diffMan(idx+spb/2:idx+spb-1) = dmLevel;

% AMI

If b == 1

AMI@ = amiPol;

amiPol = -amiPol;

else

AMI@ = 0;

End

Idx = idx + spb;

End

Codes = {unipolarNRZ, polarNRZ, polarRZ, manchester, diffMan, AMI};

Names = {'Unipolar NRZ','Polar NRZ-L','Polar RZ','Manchester',...

'Differential Manchester','AMI'};

%% Aligned waveforms

Figure('Name','Aligned Line Codes');

For k = 1:6

Subplot(6,1,k);

```

    Plot(t, codes{k}, 'LineWidth', 1.2); grid on;

    Ylim([-1.3 1.3]); ylabel(names{k});

    If k == 1, title('Line Codes for 10110010'); end

    If k == 6, xlabel('Time (bit periods)'); end

End

%%% Welch PSD

Figure('Name', 'Normalized Welch PSD'); hold on; grid on;

For k = 1:6

    [Pxx, f] = pwelch(codes{k}, hann(256), 128, 1024, fs, 'power');

    Pdb = 10*log10(Pxx/max(Pxx)+eps);

    Plot(f, Pdb, 'LineWidth', 1.2);

End

Xlim([0 3]);

Ylim([-70 3]);

Xlabel('Frequency (Hz)'); ylabel('Normalized PSD (dB)');

Title('Normalized PSD using Welch''s Method');

Legend(names, 'Location', 'best');

%%% Average level and running digital sum

Figure('Name', 'Running Digital Sum'); hold on; grid on;

For k = 1:6

    Temp = reshape(codes{k}, spb, []).';

    bitMean = mean(temp, 2);

    RDS = cumsum(bitMean);

    Plot(1:length(bits), RDS, '-o', 'LineWidth', 1.2);

```

```

End

Yline(0,'k-');

Xlabel('Bit number'); ylabel('Running digital sum');

Title('Running Digital Sum');

Legend(names,'Location','best');

%% Long run of identical bits

longBits = ones(1,20);

% Reuse a helper function below

Figure('Name','Long Run of Ones');

longCodes = {linecode_unipolar(longBits,A,spb), ...
             linecode_polar(longBits,A,spb), ...
             linecode_manchester(longBits,A,spb)};

longNames = {'Unipolar NRZ','Polar NRZ-L','Manchester'};

tt = (0:length(longBits)*spb-1)/fs;

for k=1:3

    subplot(3,1,k);

    plot(tt,longCodes{k},'LineWidth',1.2); grid on;

    ylim([-1.3 1.3]); ylabel(longNames{k});

end

xlabel('Time (bit periods)');

sgtitle('Long Run of 20 Identical 1 Bits');

%% Numerical checks

Fprintf('\nCode      Average    Power    Final RDS\n');

Fprintf('-----\n');

```

For k=1:6

Temp = reshape(codes{k},spb,[]).';

Bm = mean(temp,2);

Rds = cumsum(bm);

Fprintf('%-22s %8.4f %8.4f %8.4f\n',...

Names{k},mean(codes{k}),mean(codes{k}.^2),rds(end));

End

%% ----- Local helper functions -----

Function y = linecode_unipolar(bits,A,spb)

Y=zeros(1,length(bits)*spb); p=1;

For b=bits

Y(p:p+spb-1)=b*A; p=p+spb;

End

End

Function y = linecode_polar(bits,A,spb)

Y=zeros(1,length(bits)*spb); p=1;

For b=bits

Y(p:p+spb-1)=(2*b-1)*A; p=p+spb;

End

End

Function y = linecode_manchester(bits,A,spb)

Y=zeros(1,length(bits)*spb); p=1;

For b=bits

If $b==1$

$Y(p:p+spb/2-1)=-A; y(p+spb/2:p+spb-1)=A;$

Else

$Y(p:p+spb/2-1)=A; y(p+spb/2:p+spb-1)=-A;$

End

$P=p+spb;$

End

End

•Waveforms:

Aligned Line-Code Waveforms for 8-bit sequence 10110010

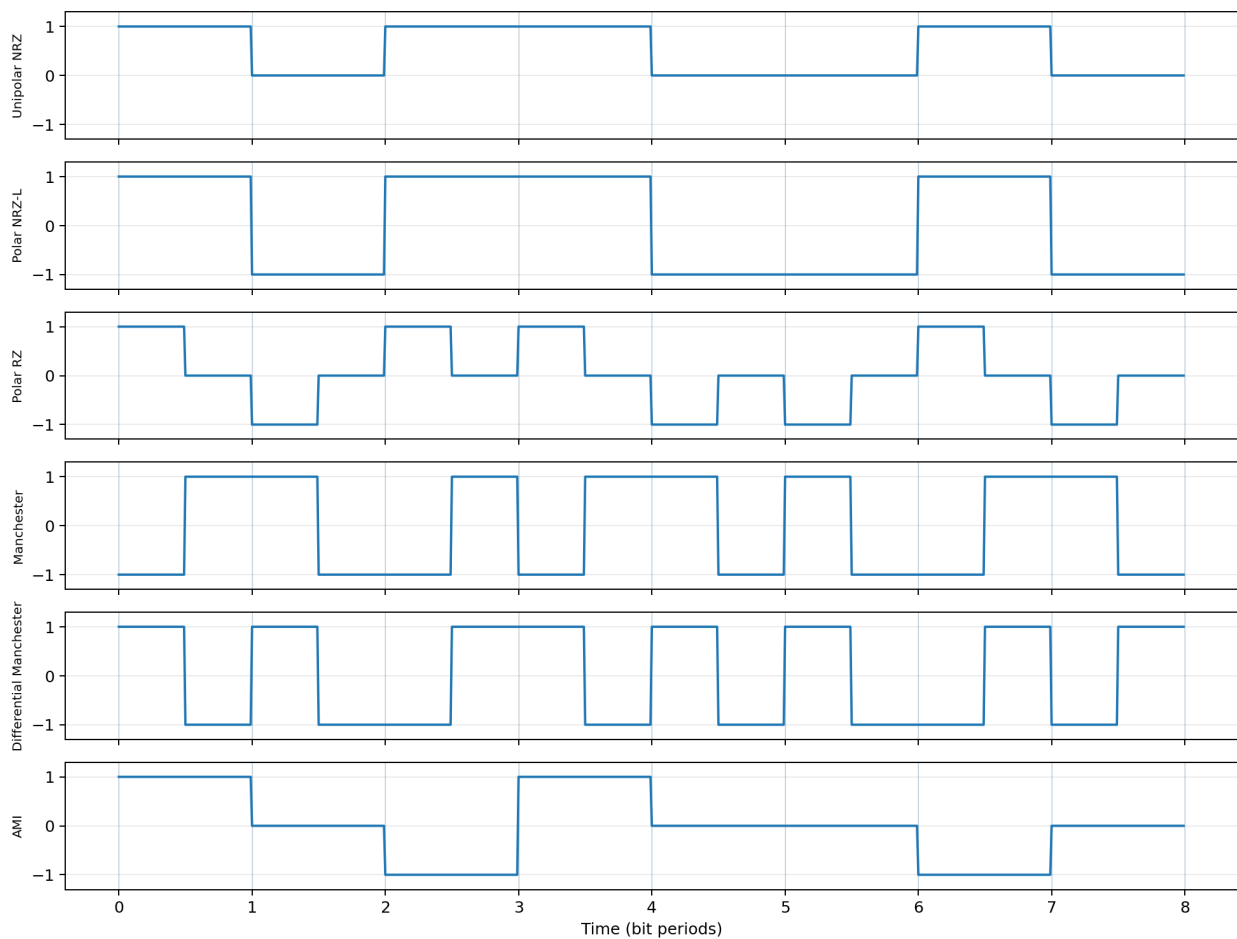


Figure 1. Aligned line-code waveforms.

•Normalized PSD using Welch's method:

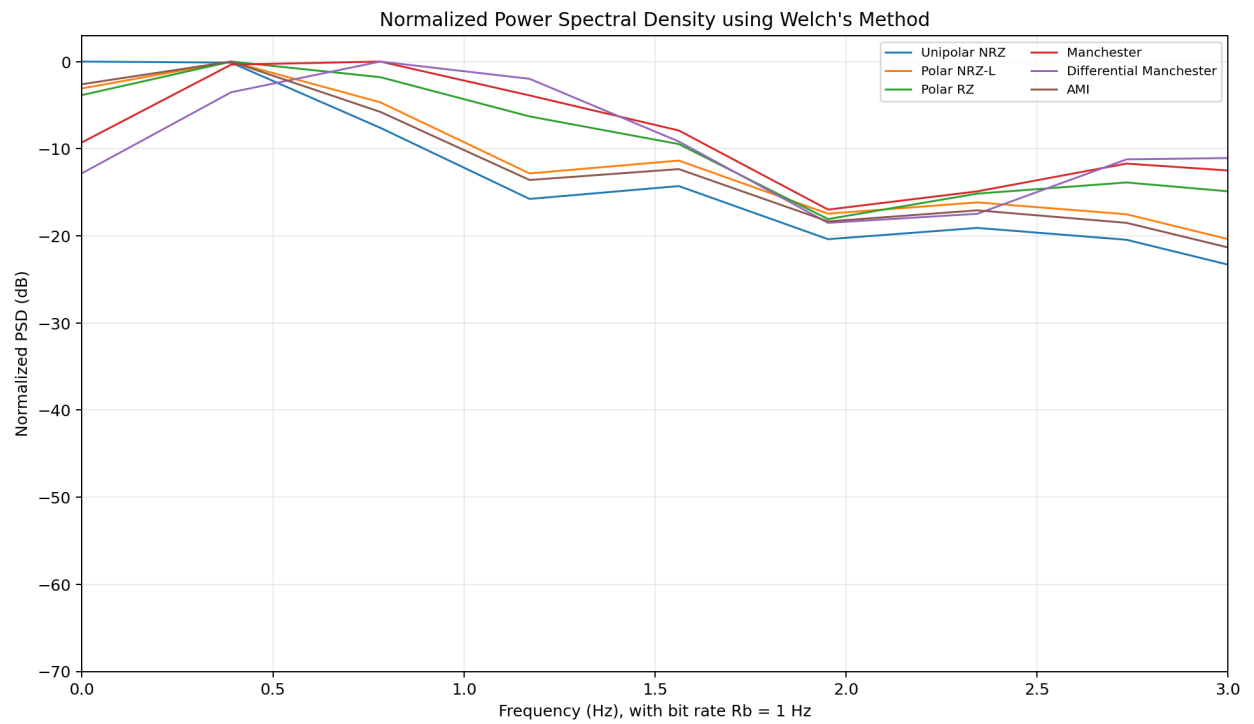


Figure 2. Normalized PSD.

•Running Digital Sum:

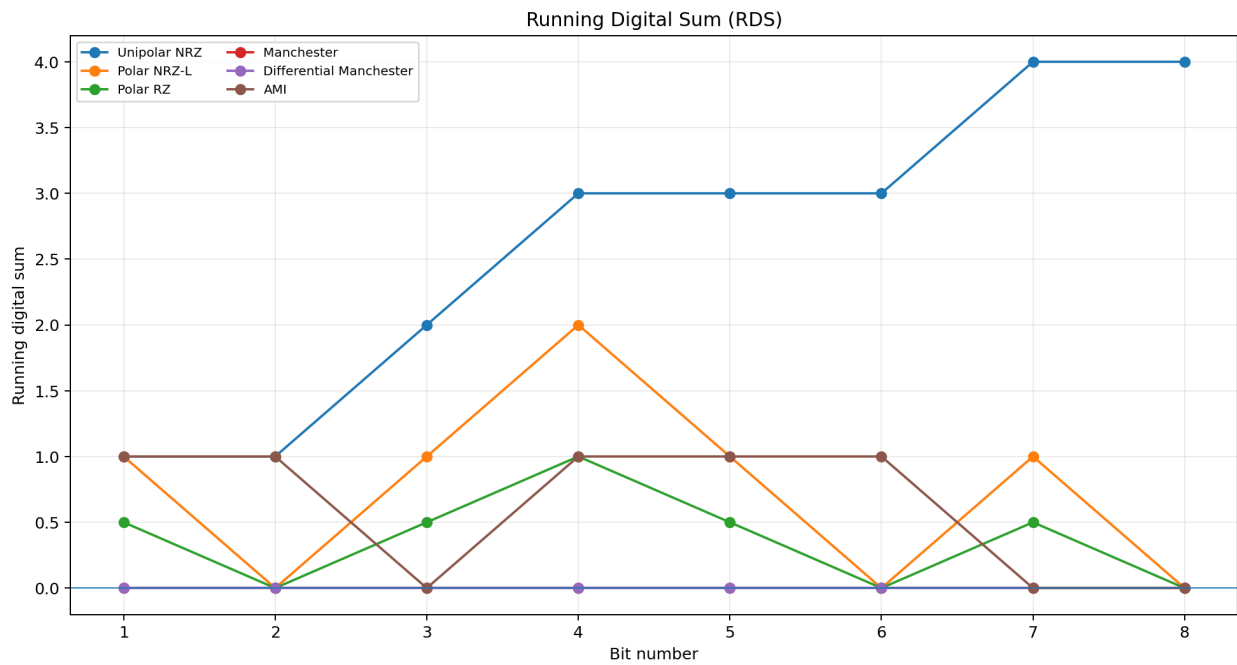


Figure 3. Running digital sum.

•Long-run test:

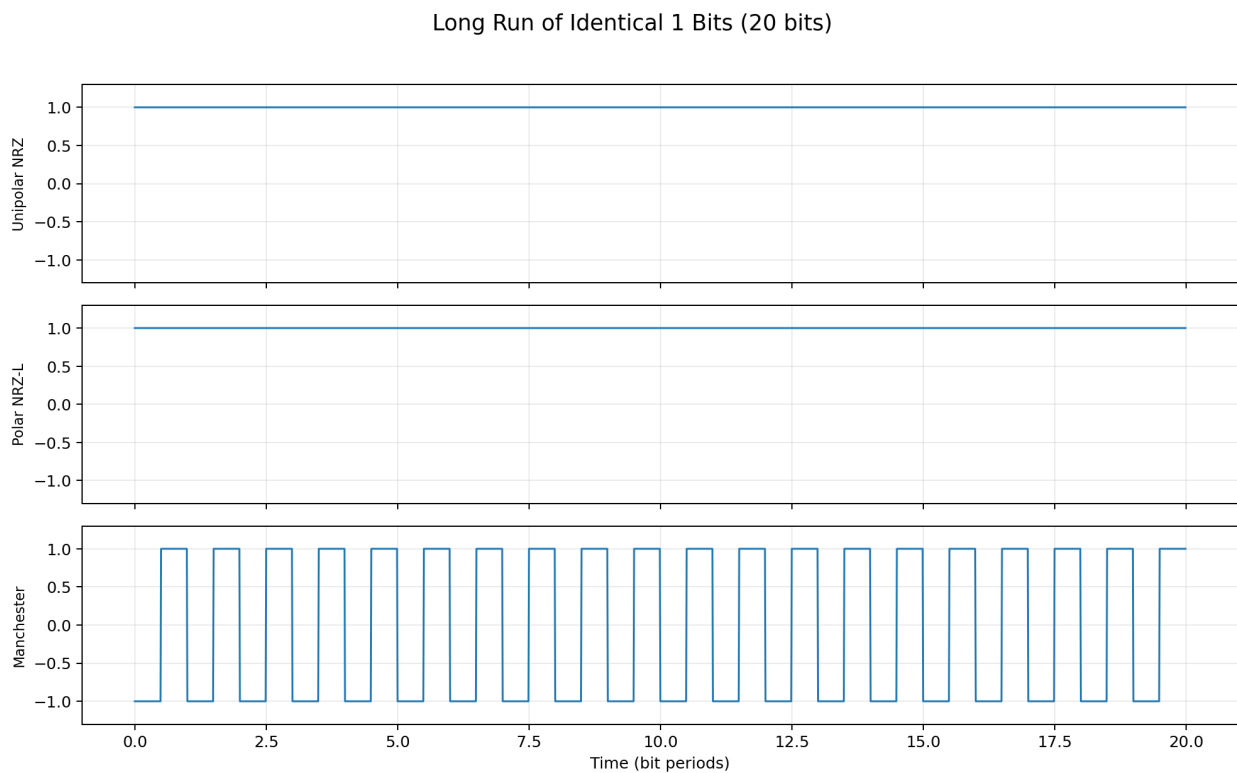


Figure 4. Long run of 20 identical 1 bits.

•**Result table:**

Line code	Average level	Average power	Final RDS
Unipolar NRZ	0.500	0.500	4.000
Polar NRZ-L	0.000	1.000	0.000
Polar RZ	0.000	0.500	0.000
Manchester	0.000	1.000	0.000
Differential Manchester	0.000	1.000	0.000
AMI	0.000	0.500	0.000

•**Interpretation:**

Expected: Unipolar NRZ should show the strongest DC tendency; polar NRZ/RZ and AMI should have reduced DC for a balanced sequence; Manchester and differential Manchester should provide the best clocking because of guaranteed mid-bit transitions. **Observed:** the waveforms and RDS agree with these expectations. The long-run test clearly shows why NRZ has poor clocking for identical bits and why Manchester maintains timing transitions. **Possible discrepancy:** the Welch PSD is based on only eight bits, so its detailed spectral shape is not identical to the theoretical infinite-random-sequence PSD. **Diagnostic test:** repeat the PSD calculation with a much longer pseudo-random sequence and compare the normalized envelope; the result should become smoother and approach the theoretical behavior.

•**Conclusion:**

The six line codes were successfully generated and compared. Manchester and differential Manchester give excellent timing recovery because of guaranteed mid-bit transitions. AMI suppresses DC effectively, while NRZ codes use less bandwidth but may lose timing information during long identical runs.

Thank you !!